

С ДЛЯ ПИТОНИСТОВ • ДЕНЬ 1

Что такое число

*Почему `2**1000` в Python считается мгновенно,
а на любом калькуляторе — переполнение?*

Вопрос дня

```
>>> 2 ** 1000
1071508607186267320948425049...
# 302 цифры, ответ точный
>>> type(2 ** 1000)
<class 'int'>
```

```
long long x = 1;
for (int i = 0; i < 1000; i++)
    x = x * 2;
printf("%lld\n", x);
```

0

Вопрос дня

```
>>> 2 ** 1000
1071508607186267320948425049...
# 302 цифры, ответ точный
>>> type(2 ** 1000)
<class 'int'>
```

```
long long x = 1;
for (int i = 0; i < 1000; i++)
    x = x * 2;
printf("%lld\n", x);
```

0

Программа печатает ноль: ни ошибки, ни исключения, ни предупреждения.

Сегодня будет показано, откуда берётся именно это число — и почему язык не обещает даже его.

РАЗДЕЛ 1

Как программа попадает в процессор

Что происходит, когда вы пишете `python a.py`

1. Запускается программа `python` — она уже скомпилирована, это машинный код.
2. Она читает ваш файл *как текст*.
3. Переводит его в байткод и *исполняет* его сама, инструкцию за инструкцией.

Ваш код на Python никогда не выполняется процессором напрямую. Его выполняет другая программа, написанная на C.

Что происходит, когда вы пишете `python a.py`

1. Запускается программа `python` — она уже скомпилирована, это машинный код.
2. Она читает ваш файл *как текст*.
3. Переводит его в байткод и *исполняет* его сама, инструкцию за инструкцией.

Ваш код на Python никогда не выполняется процессором напрямую. Его выполняет другая программа, написанная на C.

Переход на другую сторону

Дальше вы пишете программы, которые процессор исполняет нативно. Между вашим кодом и оборудованием не остаётся промежуточного слоя, защищающего от ошибки.

Первая программа

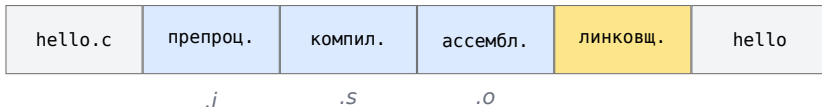
```
#include <stdio.h>

int main(void) {
    printf("Hello, machine\n");
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra hello.c -o hello
$ ./hello
Hello, machine
$ echo $?
0
```

- `main` — точка входа. Её вызывает операционная система.
- `return 0` — код возврата. Ноль значит «успех»; это то, что видит `$?` и любой скрипт, запустивший вашу программу.
- `#include` — *не* `import`.

Шаги от текста к машинному коду



```
$ gcc -E hello.c -o hello.i    # препроцессор: подставил stdio.h целиком
$ gcc -S hello.c -o hello.s    # компилятор:   получился ассемблер
$ gcc -c hello.c -o hello.o    # ассемблер:   машинный код, но без printf
$ gcc hello.o -o hello         # линковщик:   приделал printf из libc
```

Препроцессор — это простая подстановка текста

```
$ wc -l hello.c          6 hello.c
$ gcc -E hello.c | wc -l 782 hello.i
```

`#include <stdio.h>` вставил в файл 776 строк чужого текста. Ни пространств имён, ни модулей — просто подстановка текста.

Как это выглядит в Python

`import math` создаёт объект-модуль в памяти и кладёт его в переменную. `#include <math.h>` вклеивает текст с *объявлениями* функций. Сами функции приделает линковщик на последнем шаге.

Поэтому `gcc prog.c -lm` иногда нужен, а `import math` — никогда: объявления и реализация в С лежат в разных местах.

Эти файлы лежат у вас на диске

Компилятор ищет `<...>` в нескольких каталогах:

```
$ echo | gcc -xc -E -v - 2>&1 | grep -A3 "search starts"
#include <...> search starts here:
/usr/lib/gcc/x86_64-redhat-linux/15/include
/usr/local/include
/usr/include
```

Флаг `-H` печатает дерево: кто кого включил (точки — глубина).

```
$ gcc -E -H hello.c -o /dev/null
. /usr/include/stdio.h
.. /usr/include/bits/libc-header-start.h  # и ещё 30 файлов
```

Проверьте самостоятельно

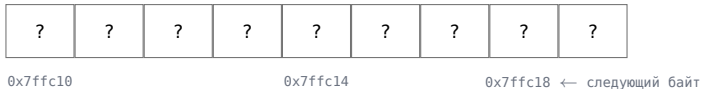
Откройте `/usr/include/stdio.h` и найдите там `printf`. Одна строка, тела функции нет: реализация лежит в `libc.so`.

РАЗДЕЛ 2

Память

Память — это один длинный массив байтов

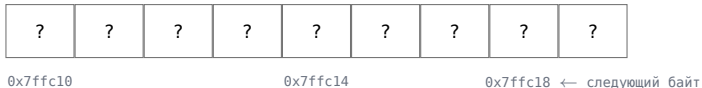
Байт — минимально адресуемая ячейка памяти: самый маленький кусок со своим номером, 8 бит (обычно). У отдельного бита адреса нет — чтобы добраться до бита, процессор читает байт целиком.



- Номер байта — это **адрес**; пишут его шестнадцатеричным.
- Ни типов, ни имён, ни границ объектов: всё это существует только в компиляторе.

Память — это один длинный массив байтов

Байт — минимально адресуемая ячейка памяти: самый маленький кусок со своим номером, 8 бит (обычно). У отдельного бита адреса нет — чтобы добраться до бита, процессор читает байт целиком.



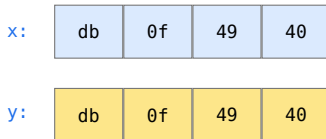
- Номер байта — это **адрес**; пишут его шестнадцатеричным.
- Ни типов, ни имён, ни границ объектов: всё это существует только в компиляторе.

Главное

В памяти нет «списка», «строки» и «числа». Есть байты и *соглашение* о том, как их читать. Тип — это и есть соглашение.

Тип = сколько байт + как их понимать

```
int    x = 1078530011;  
float  y = 3.1415927f;
```



Байты одинаковые. Различаются только инструкции, которыми процессор их читает: целочисленные для `x`, вещественные для `y`.

```
printf("%zu %zu\n", sizeof(int), sizeof(float)); // 4 4
```

Целые типы и их размеры

Тип	Обычно байт	Гарантия стандарта	Диапазон (чаще всего)
char	1	ровно 1	$-128 \dots 127$
short	2	≥ 2	$-32768 \dots 32767$
int	4	≥ 2	$-2^{31} \dots 2^{31} - 1$
unsigned int	4	столько же, сколько int	$0 \dots 2^{32} - 1$
long	8	≥ 4	$-2^{63} \dots 2^{63} - 1$
long long	8	≥ 8	$-2^{63} \dots 2^{63} - 1$
size_t	8	вмещает размер любого объекта	$0 \dots 2^{64} - 1$

Любой целый тип бывает `unsigned`: битов столько же, но знаковый разряд работает как обычный, и весь диапазон уезжает в положительную сторону. **Стандарт не обещает, что `int` — это 4 байта**; когда размер важен — берите точные типы из `<stdint.h>`:

```
int32_t a; // ровно 32 бита со знаком
uint32_t b; // ровно 32 бита без знака
```

Вещественные типы: float и double

Тип	Байт	Верных десятичных цифр	Что это
float	4	6	IEEE 754 одинарной точности
double	8	15	IEEE 754 двойной точности
long double	16	18	расширенная, только на x86

Как это выглядит в Python

float в Python — это в точности double из C, отсюда все знакомые странности:

```
printf("%.20f\n", 0.1 + 0.2);    // 0.3000000000000000004441
```

«Верных цифр» — не то же, что «диапазон»: float доходит до $3.4 \cdot 10^{38}$, но теряет точность на 16 777 217:

```
float f = 16777217.0f;
printf("%.1f\n", f);              // 16777216.0
```

Двоичная запись

```
uint8_t v = 37;
```

0	0	1	0	0	1	0	1
128	64	32	16	8	4	2	1

$32 + 4 + 1 = 37$. Старший бит слева, как в обычной записи числа.

Шестнадцатеричная запись — это та же двоичная, сгруппированная по 4 бита:

$$0010\,0101 = 0x25 = 37$$

Поэтому адреса и дампы памяти всегда печатают в hex: два символа — ровно один байт.

Отрицательные числа: пример кодирования

Как записать -1 , если в ячейке только нули и единицы?

Правило: $-x$ хранится как $2^n - x$. Для 8 бит:

$$-1 \rightarrow 256 - 1 = 255 \rightarrow 1111\,1111$$

$$-2 \rightarrow 256 - 2 = 254 \rightarrow 1111\,1110$$

```
int8_t  a = -1;
uint8_t b = (uint8_t)a;
printf("%d\n", b);      // 255 -- те же биты, другое соглашение
```

Отрицательные числа: пример кодирования

Как записать -1 , если в ячейке только нули и единицы?

Правило: $-x$ хранится как $2^n - x$. Для 8 бит:

$$-1 \rightarrow 256 - 1 = 255 \rightarrow 1111\,1111$$

$$-2 \rightarrow 256 - 2 = 254 \rightarrow 1111\,1110$$

```
int8_t  a = -1;
uint8_t b = (uint8_t)a;
printf("%d\n", b);      // 255 -- те же биты, другое соглашение
```

Зачем так? Потому что тогда **сложение работает без изменений**: процессору не нужна отдельная схема для отрицательных.

$$1111\,1111 \text{ } (-1) + 0000\,0001 \text{ } (1) = 1\,0000\,0000 \rightarrow 0000\,0000 \text{ } (0)$$

Девятый бит просто выпал за границу байта.

Переполнение: возвращаемся к вопросу дня

```
long long x = 1;
for (int i = 0; i < 1000; i++) x = x * 2;
```

После 63 удвоений единственный единичный бит уходит за границу 64 бит: процессор умножает по модулю 2^{64} , и в переменной остаётся ноль.

2^0 :

00	00	00	00	00	00	00	01
----	----	----	----	----	----	----	----

2^{63} :

80	00	00	00	00	00	00	00
----	----	----	----	----	----	----	----

2^{64} :

00	00	00	00	00	00	00	00
----	----	----	----	----	----	----	----

бит вышел за границу типа

Так работает процессор. **Язык не обещает даже этого:** знаковое переполнение — неопределённое поведение, и ноль здесь один из результатов, а не гарантирован.

Как с этим справляется Python

Python int — это структура, а не примитивный тип

```
>>> import sys
>>> sys.getsizeof(0), sys.getsizeof(2**1000)
(28, 160)
```

Внутри CPython `int` — это объект, в котором лежит **массив 30-битных «цифр»** переменной длины. Сложение больших чисел — цикл по этому массиву, ровно как сложение столбиком в школе.

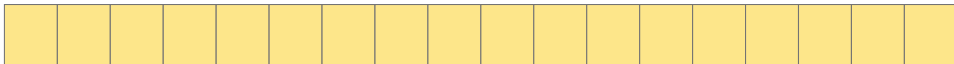
- 2^{1000} занимает 160 байт вместо 8 — **в 20 раз больше**.
- Сложение — не инструкция процессора, а вызов функции с циклом.
- Зато не переполняется никогда: не хватило места — выделили ещё.

Чем Python платит за отсутствие переполнения

C:

8 байт

Py:



160 байт: заголовок объекта и массив 30-битных «чисел»

Главное

Python платит памятью и скоростью за то, чтобы о переполнении не приходилось думать.

В C этой цены нет, и потому контроль за переполнением остаётся за автором программы.

Переполнение знакового — это UB

```
int x = INT_MAX;  
x = x + 1;           // не станет "INT_MIN". Это UB.
```

Неопределённое поведение

Undefined behaviour — стандарт не описывает, что произойдёт. Компилятору разрешено всё: любой результат, падение, выкидывание кода.

Переполнение знакового — это UB

```
int x = INT_MAX;  
x = x + 1;           // не станет "INT_MIN". Это UB.
```

Неопределённое поведение

Undefined behaviour — стандарт не описывает, что произойдёт. Компилятору разрешено *всё*: любой результат, падение, выкидывание кода.

```
for (int i = 1; i > 0; i++) { /* ... */ }
```

Компилятор рассуждает так: «переполнение знакового невозможно $\Rightarrow$ `i` только растёт $\Rightarrow$ `i > 0` всегда истинно» — и выбрасывает проверку. Цикл становится бесконечным: с `-00` программа завершится, с `-02` — нет. **Компилятор всегда исходит из того, что UB в вашей программе нет.**

А беззнаковое переполнение — определено

```
unsigned int u = UINT_MAX;  
u = u + 1;           // строго определено: станет 0
```

Для `unsigned` стандарт обещает арифметику по модулю 2^n . Так и делают намеренно — в хешах, генераторах случайных чисел, циклических буферах.

Упрощённое правило

Считаете что-то, что переполняется намеренно (хеш, контрольная сумма) — берите `unsigned`. В остальных случаях переполнение — ваша ошибка, и `-fsanitize=undefined` о ней скажет.

```
$ gcc -fsanitize=undefined ovf.c -o ovf && ./ovf  
ovf.c:5:9: runtime error: signed integer overflow:  
2147483647 + 1 cannot be represented in type 'int'
```

РАЗДЕЛ 3

Ловушки при переходе от Python к C

Деление: C и Python дают разные ответы

```
>>> -7 // 2
-4
>>> -7 % 2
1
```

Python округляет **вниз** (к $-\infty$).

```
printf("%d\n", -7 / 2); // -3
printf("%d\n", -7 % 2); // -1
```

C округляет **к нулю**.

Где это проявляется

Хеш-функция `h % size` в C может вернуть *отрицательный* индекс. Обращение по нему — выход за границы массива и, скорее всего, падение.

Поэтому в C для остатка почти всегда берут `unsigned`: тогда результат гарантированно неотрицательный.

Три различия, о которых Python позволял не думать

1. **Возведения в степень нет.** Запись `2 ** 10` разбирается как `2 * (*10)`, то есть как разыменование числа:

```
error: invalid type argument of unary '*' (have 'int')
```

Нужен цикл или `pow()` из `<math.h>`, возвращающая `double`.

Три различия, о которых Python позволял не думать

1. **Возведения в степень нет.** Запись `2 ** 10` разбирается как `2 * (*10)`, то есть как разыменованное число:

```
error: invalid type argument of unary '*' (have 'int')
```

Нужен цикл или `pow()` из `<math.h>`, возвращающая `double`.

2. **/ между целыми всегда целое.** `1 / 2` равно 0, а не 0.5. Для дроби нужно приведение: `1.0 / 2` или `(double)a / b`.

Три различия, о которых Python позволял не думать

1. **Возведения в степень нет.** Запись `2 ** 10` разбирается как `2 * (*10)`, то есть как разыменованное число:

```
error: invalid type argument of unary '*' (have 'int')
```

Нужен цикл или `pow()` из `<math.h>`, возвращающая `double`.

2. **/ между целыми всегда целое.** `1 / 2` равно 0, а не 0.5. Для дроби нужно приведение: `1.0 / 2` или `(double)a / b`.
3. **Сравнение цепочкой не работает.** `if (0 <= i <= n)` вычисляет `(0 <= i)`, получает 0 или 1 и сравнивает результат с `n`. Правильная запись — `if (0 <= i && i <= n)`.

Первая запись не компилируется, третью `gcc` с `-Wall` отмечает предупреждением `-Wparentheses`. **Вторая проходит молча** и остаётся самой опасной из трёх.

Битовые операции

Оп.	Смысл	Пример	Зачем
<code>&</code>	И	<code>x & 1</code>	чётность
<code> </code>	ИЛИ	<code>x 8</code>	поднять бит
<code>^</code>	исключ. ИЛИ	<code>x ^ y</code>	обмен, хеши
<code>~</code>	инверсия	<code>~x</code>	все биты наоборот
<code><<</code>	сдвиг влево	<code>1 << k</code>	2^k
<code>>></code>	сдвиг вправо	<code>x >> 1</code>	деление на 2

Три способа получить UB сдвигом

- `1 << 32` — сдвиг на всю ширину 32-битного типа и больше.
- `1 << -1` — отрицательное количество.
- `-1 >> 1` — сдвиг отрицательного вправо зависит от реализации.

Нужно сдвигать на 32 и больше — берите `1ULL << 40`.

РАЗДЕЛ 4

Основы синтаксиса

Знакомые конструкции почти не меняются

```
def gcd(a, b):  
    while b != 0:  
        a, b = b, a % b  
    return a  
  
n = int(input())  
for i in range(n):  
    if i % 3 == 0:  
        print(i)
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}  
  
int n;  
scanf("%d", &n);  
for (int i = 0; i < n; i++)  
    if (i % 3 == 0)  
        printf("%d\n", i);
```

Отличия: типы у всего, скобки вместо отступов, точка с запятой.

printf: спецификатор обязан совпасть с типом

%d	int	%f	double
%u	unsigned int	%c	один символ
%ld	long	%s	строка, char *
%lld	long long	%p	адрес
%zu	size_t	%x / %X	hex, строчными / прописными

Буквы перед типом — это *длина* аргумента: `l` — `long`, `ll` — `long long`, `z` — `size_t`. Без них `printf` возьмёт из стека 4 байта вместо 8.

Неопределённое поведение

```
long long big = 50000000000;  
printf("%d\n", big);    // UB: обещали int, дали 8 байт
```

`printf` не знает типов своих аргументов — он верит строке формата, и только ей. **-Wall ловит это почти всегда.**

printf: ширина, нули, точность

Между % и буквой типа есть специальные модификаторы:

Запись	Что делает
%8d	занять не меньше 8 позиций, прижать вправо
%-12s	не меньше 12 позиций, прижать <i>влево</i>
%2zu	не меньше 2 — чтобы колонка не плясала
%02X	2 позиции, добить <i>нулями</i> , hex прописными
%.20f	ровно 20 знаков после запятой

Главное

Ширина — это *минимум*, а не обрезание: значение длиннее поля напечатается целиком, и колонка всё-таки поедет. А вот точность .N у %f — это ровно N знаков, ни больше ни меньше.

scanf: ЧИТАЕМ ВВОД

```
int n;  
if (scanf("%d", &n) != 1) { /* на входе было не число */ }
```

- `&n` — «положить прочитанное вот сюда». Что такое `&`, разберём позже; пока просто ставьте его перед каждой переменной.
- `scanf` возвращает **количество прочитанных значений**, а не само значение. `scanf("%d %d", &a, &b)` вернёт 2, если удалось прочитать оба, 1 — если только первое, и 0 или `E0F` — если ничего.
- Отсюда идиоматика «читать до конца ввода»:

```
while (scanf("%d", &n) == 1) { ... }
```

Отличие от printf

Для `double` у `printf` формат `%f`, а у `scanf` — `%lf`. `scanf("%f", &x)` с переменной типа `double` запишет 4 байта вместо 8.

Как собираются задачи курса

```
$ gcc -std=c11 -Wall -Wextra -g -O0 \  
    -fsanitize=address,undefined prog.c -o prog
```

- `-Wall -Wextra` — предупреждения. **Предупреждение в C почти всегда означает ошибку.**
- `-g` — отладочная информация: в отчётах будут имена файлов и номера строк.
- `-fsanitize=address` — ловит выход за границы и работу с освобождённой памятью.
- `-fsanitize=undefined` — ловит переполнение, кривые сдвиги, деление на ноль.

Всё это уже прописано в `common/task.mk`: в каталоге задач достаточно набрать `make`. Санитайзеры замедляют программу вдвое, поэтому на замерах скорости собирайте через `make release` — там их нет.

Итог дня

- Память — массив байтов. Тип — соглашение о том, сколько байт брать и как их толковать.
- Целые типы имеют фиксированный размер, поэтому **переполняются**. Знаковое переполнение — UB, беззнаковое — арифметика по модулю.
- Python не переполняется, потому что его `int` — массив цифр переменной длины. Это стоит памяти и скорости.
- `-7 / 2` в C равно `-3`, а не `-4`. Остаток может быть отрицательным.
- Предупреждение компилятора означает, что программа, скорее всего, выполнит не то, что предполагал автор.

Задачи дня: `day1-memory-model/tasks/`

Вопросы к защите открыты: `defense/questions.md`, посмотрите их до того, как начнёте решать.